Layer 2 attack & defense configuration

Alex Huhdanpaa

Purpose:

The purpose of this lab was to learn how to attack and protect at layer 2. From this lab it demonstrated how to do a CAM Table Overflow attack, a ARP spoofing attack, and a STP Manipulation attack. For these attacks we used Linux to run the different attacks. The goal is to learn how to do these attacks using Linux, and the commands used in the switch to prevent those attacks.

Background Information:

When it comes to network security, a key worry in local area networks (LANs) is how bad guys can mess with Layer 2 protocols—in switched setups. These networks are safer than shared stuff like hubs, but they can still be tricked by people up to no good. Some of the big ways to attack include messing with Spanning Tree Protocol (STP) overflowing CAM tables, and faking ARP. Each of these attacks goes after different parts of Layer 2, and network folks can fight back with specific defenses. These include BPDU Guard, Port Security, and Dynamic ARP Inspection (DAI), which often team up with other tricks like DHCP Snooping. To keep networks safe and strong, it's key to know how these attacks work and how to stop them.

The Spanning Tree Protocol (STP) aims to stop loops in a network by creating a loop-free logical layout. Switches in an STP-enabled network swap Bridge Protocol Data Units (BPDUs) to pick a root bridge—the main switch that shapes path choice across the network. , the root bridge is the switch with the lowest bridge ID, which mixes priority and MAC address. But this method can be messed with. Bad guys can use tools like Yersinia, a strong Layer 2 attack tool, to send fake BPDUs that claim very low bridge IDs. If this works, it can fool other switches into thinking the attacker's device is the new root bridge. Once a hacker's system becomes the root bridge, it can send traffic through itself. This lets the hacker listen in, change traffic, or cause service outages by making the network layout unstable.

Network admins can protect against this attack type by turning on BPDU Guard for access ports—those linked to end-user devices instead of other switches. BPDU Guard is a security tool that turns off a port right away when it gets unexpected BPDUs. Because real end devices should never take part in STP by sending BPDUs, getting these packets points to a possible attack or wrong setup. When BPDU Guard is on and a fake device like one running Yersinia tries to change STP, the switch shuts down the affected port at once. This stops the attack before it can mess up the topology. BPDU Guard works well when used with PortFast, which lets ports move straight into the forwarding state when connected, but still makes sure any BPDU will cause a shutdown. This easy but strong defense keeps the STP election process safe and makes sure approved switches help decide the network's layout.

A weak spot in Layer 2 networks is the Content Addressable Memory (CAM) table overflow. This attack targets how switches link MAC addresses to physical ports. Switches keep a CAM table to send frames to the right port, which boosts both speed and safety. But hackers can use tools like macof—from the dsniff toolkit—to swamp the switch with tons of frames that have fake source MAC addresses. They want to use up all the CAM table's memory. This forces the switch into a backup mode where it sends all incoming frames out of every port just like a hub would. As a result, privacy takes a big hit. Any attacker on the same switch can now get traffic they shouldn't see, which lets them snoop on packets and steal data.

Port Security stands as the main defense against CAM table overflow attacks. Most managed switches offer this feature. Network admins can limit the number of MAC addresses that switches learn by turning on Port Security for switch ports. When the limit is hit and the switch spots another MAC address in restrict mode, it blocks the problematic traffic and records the breach. This approach doesn't shut down the whole port making it less harmful than shutdown mode. It helps stop the attack while keeping some service running. You can also set up Port Security to allow certain MAC addresses on a port, or to cap the number of addresses learned on the fly and protect them from changes. When set up right, this method stops macof and similar tools from working. The flood of fake MAC addresses will either get ignored or logged as violations. This keeps the switch working and protects the privacy of network data.

A third and sneaky attack is ARP spoofing or ARP poisoning. This attack takes advantage of the Address Resolution Protocol that devices on a local network use to link IP addresses to MAC addresses. ARP trusts by default and doesn't keep track of past interactions. Any device can answer an ARP request with any MAC address. This flaw in design lets attackers use tools like dsniff to send fake ARP replies. These replies connect the IP address of another host (like the gateway or a target machine) to the attacker's MAC address. As a result, the attacker catches traffic meant for another device setting up man-in-the-middle (MitM) attacks. They can look at, change, or stop packets, which puts data safety and privacy at risk.

To guard against ARP spoofing, you can set up modern switches with Dynamic ARP Inspection (DAI). This security feature checks the validity of ARP packets on the network. DAI teams up with DHCP Snooping to create a trusted database of IP-to-MAC pairings based on real DHCP exchanges. When you turn on DAI, the switch compares every ARP packet to this trusted binding table. If a packet has an IP-MAC pairing that doesn't match the DHCP Snooping records, the switch blocks it stopping ARP spoofing tries. This check makes sure the network accepts and sends on genuine ARP responses, which cuts down the risk of MitM attacks. You can apply DAI just to untrusted ports those facing end-user devices. Trusted ports, like uplinks to other gear, don't need this check to avoid false alarms.

Summary:

In this lab there are 3 checkoffs, one per attack. We started with the STP Manipulation attack where we used Yersinia to steal the root bridge position and therefore be able to see all packets sent. We then BPDU guard on the switch to shut down the port that was attacking the network. Next, we used CAM Table Overflow attack using the Linux program called macof, which sent thousands of different MAC addresses to the port which the computer was connected. To protect against this we enabled port security and shut down the port that was sending the addresses to properly allow routing. For the final attack, we used nmap to find active ips and then used the arpspoof commands on dsniff to send fake arp messages between the router and PC. To defend against this attack we used DHCP snooping and dynamic arp inspection to prevent the fake arp messages.

Commands:

At first we attempted VLAN hopping, however we couldn't figure it out. Traffic was stopped when the mac addresses overflowed the CAM table and the switch was unconfigurable due to error messages which we should've planned for and hid the messages. We also didn't enable dynamic arp inspection when we first tried the arp poisoning attack and therefore was unable to fix the problem.

Configurations:

In conclusion, we learned how to attack and protect a few of the common layer 2 attacks. Mainly, we learned how to install and attack networks using Linux and also how to configure a network to protect against common layer 2 attacks. We learned how to use Linux programs to attack using dsniff, yersinia and nmap. We most importantly learned how to defend against these attacks using arp inspection, port security and BPDU guard.

Attacking Computer:

```
sudo apt install yersinia
sudo yersinia -i
sudo apt install dsniff
sudo macof -i eno2
sudo apt install dsniff
sudo arpspoof -i eno2 -t 192.168.0.1 <target ip>
sudo arpspoof -i eno2 -t <target ip> 192.168.0.1
sudo apt install nmap
nmap 192.168.0.0/24
```

Router 1:

```
interface g0/0/0
ip address 192.168.0.1 255.255.255.0
no shutdown
ip dhcp pool POOL
network 192.168.0.0 255.255.255.0
default-router 192.168.0.1
ip dhcp excluded-address 192.168.0.1
```

Switch 1:

```
interface range f0/2-24
switchport mode access
switchport port-security
switchport port-security maximum 1
switchport port-security violation restrict
interface range fa0/1 - 24
switchport mode access
spanning-tree portfast
spanning-tree bpduguard enable
ip dhcp snooping
ip dhcp snooping vlan 1
interface f1/0/1
ip dhcp snooping trust
exit
interface range f1/0/2-24
ip dhcp snooping limit rate 10
exit
ip arp inspection vlan 1
int f1/0/1
ip arp inspection trust
exit
```

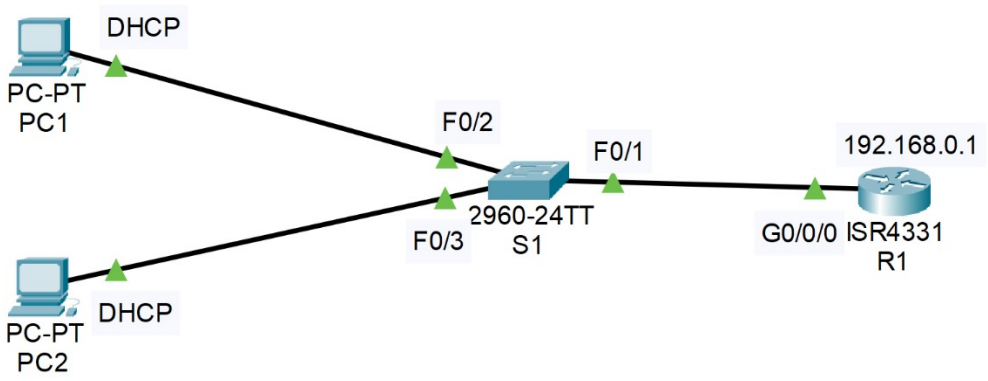
Problems:

At first when we attempted to do a CAM overflow attack we were unable to successfully spy on unicast pings between the attacker and default gateway, when we looked more into it realized we were using switches that protected unicast packets from being broadcast – even during a CAM overflow attack, so that idea quickly went away, our second issue was that we were unable to stop the ARP spoofing attack, even with enabling DAI, we found this issue was due to the fact that there were preexisting entries that needed to be deleted in the router and victim devices, as they still had malicious entries in their tables that couldn't be removed by DAI as they were already in there, as DAI only governs new entries, therefore we had to delete these entries and then and only then were we able to stop the attacker from spoofing as a default gateway/end user

Conclusion:

In conclusion, we learned how to attack and protect a few of the common layer 2 attacks. Mainly, we learned how to install and attack networks using Linux and also how to configure a network to protect against common layer 2 attacks. We learned how to use Linux programs to attack using dsniff, yersinia and nmap. We most importantly learned how to defend against these attacks using arp inspection, port security and BPDU guard.

Diagram and Addressing:



Signoff:

CCNP Layer 2 Attacks

Alex Huhdanpaa